

PER2-PROTAC: QC PSM-level quantification and summarisation to protein-level abundance

Tom Smith

2026-01-30

Here, we will read in the PSM-level quantification data, perform routine PSM filtering and then summarise the quantification to protein-level abundances.

Setup

Load required packages

```
library(biomasslmb)
library(ggplot2)
library(tidyr)
library(dplyr)
library(here)
library(QFeatures)
library(openxlsx)
```

Read in Uniprot protein details

```
uniprot2details_unique <- readRDS(here('external/uniprot2details_unique.rds'))
```

Defining the contaminant proteins

We need to remove contaminant proteins. Below, we parse the MaxQuant contaminants fasta to extract the IDs for the proteins.

```
contaminant_proteins <- get_maxquant_cont_accessions()
```

```
## No contaminants fasta supplied, defaulting to contaminants.fasta.gz from biomasslmb
```

Define the experiment design

We can extract the details for the experiment to sample mapping from the Excel file provided.

```
exp_design <- openxlsx::read.xlsx(here('raw/PROTAC/exp_design.xlsx'),
                                startPos = 7, colNames = FALSE) %>%
  setNames(c('sample_name', 'sample_conditions', 'reporter_n',
            'tag', 'reporter_mass')) %>%
  separate(sample_conditions, into=c('genotype', 'condition', 'replicate'),
            sep=' ', remove = FALSE) %>%
  mutate(genotype=ifelse(genotype=='P1H/P2H', 'P1H.P2H', genotype)) %>%
  mutate(quantCols=reporter_n)
```

Read in input data

We start by reading in quantitative proteomics data into a **QFeatures** object, which is the standard Bioconductor object for holding quantitative proteomics data. See [here](#) for documentation about the **QFeatures** object.

```
pg_inf <- here('raw/PROTAC/proteinGroups.txt.gz')
pg_df <- read.delim(pg_inf)
table(pg_df$Q.value<0.01)
```

```
##
## FALSE TRUE
## 548 8847
```

Creating the QFeatures object

Now we can now create the QFeatures object. Here, we will create two PSM-level QFeatures objects, one for the total proteome and one for the phosphoproteome.

```
psm_inf <- here('raw/PROTAC/evidence.txt.gz')

infd_f <- read.delim(psm_inf)

# Merge on the leading razor protein column from the Peptides file
infd_f <- merge(infd_f, uniprot2details_unique,
               by.x='Leading.razor.protein', by.y='UniprotID', all.x=TRUE) %>%
  merge(pg_df[,c('Majority.protein.IDs', 'Q.value')], 'Leading.razor.protein',
        by.y='Majority.protein.IDs', all.x=TRUE)

# Define the abundance columns
abundance_cols <- 56:73

# Rename the abundance columns to be just the reporter mass
colnames(infd_f)[abundance_cols] <- gsub(
  'Reporter.intensity.corrected.', '', colnames(infd_f)[abundance_cols])

# Split into phospho and total
phospho_infd_f <- filter(infd_f, grepl('phos', Raw.file))
total_infd_f <- filter(infd_f, !grepl('phos', Raw.file))

# Create QFeatures object for total
```

```

tmt_qf <- QFeatures::readQFeatures(
  assayData = total_infdf,
  quantCols = abundance_cols,
  colData = exp_design, name='psm_raw')

## Checking arguments.

## Loading data as a 'SummarizedExperiment' object.

## Formatting sample annotations (colData).

## Formatting data as a 'QFeatures' object.

# Copy over the colData
colData(tmt_qf[['psm_raw']]) <- colData(tmt_qf)

# Create QFeatures object for phospho
phospho_tmt_qf <- QFeatures::readQFeatures(
  assayData = phospho_infdf,
  quantCols = abundance_cols,
  colData = exp_design, name='psm_raw')

## Checking arguments.

## Loading data as a 'SummarizedExperiment' object.

## Formatting sample annotations (colData).

## Formatting data as a 'QFeatures' object.

colData(phospho_tmt_qf[['psm_raw']]) <- colData(phospho_tmt_qf)

# Combine into a list and replace zeroes with NAs
qfs <- list('total'=tmt_qf, 'phospho'=phospho_tmt_qf)
qfs <- lapply(qfs, function(x) zeroIsNA(x, names(x)))

Subset to the PER2-PROTAC samples only

qfs <- lapply(qfs, function(x) x[,colData(x)$genotype == 'P2H'])

```

Filtering PSMs

We first perform routine filtering to remove PSMs that:

- Could originate from contaminants. See `?filter_features` for further details, including the removal of ‘associated cRAP’.
- Don’t have a unique master protein.

```

for(qf in names(qfs)){

  in_assay_name <- 'psm_raw'
  out_assay_name <- 'psm_filtered'

  qfs[[qf]][[out_assay_name]] <- filter_features_mq_dda(
    qfs[[qf]][[in_assay_name]],
    contaminant_proteins=contaminant_proteins,
    filter_contaminant=TRUE,
    filter_associated_contaminant=TRUE,
    unique_master=TRUE)

  if(qf == 'total'){
    qfs[[qf]] <- filterFeatures(qfs[[qf]], i=out_assay_name, ~ Q.value <= 0.01)

    biomasslmb:::message_parse(rowData(qfs[[qf]][[out_assay_name]]),
                              "Leading.razor.protein",
                              "Proteins with Q-Value > 0.01 removed")
  }
}

```

Filtering data...

113093 features found from 8177 master proteins => Input

112923 features found from 8060 master proteins => Removed hits to decoy database

246 contaminant proteins supplied

1318 proteins identified as 'contaminant associated'

111841 features found from 7994 master proteins => contaminant features removed

110711 features found from 7962 master proteins => associated contaminant features removed

110711 features found from 7962 master proteins => MaxQuant-labelled 'Contaminants' removed

110711 features found from 7962 master proteins => features without a master protein removed

106115 features found from 7940 master proteins => features with non-unique master proteins removed

85996 features found from 7806 master proteins => features without quantification removed

'Q.value' found in 2 out of 2 assay(s).

85931 features found from 7748 master proteins => Proteins with Q-Value > 0.01 removed

Filtering data...

```
## 64807 features found from 6814 master proteins => Input

## 64532 features found from 6611 master proteins => Removed hits to decoy database

## 246 contaminant proteins supplied

## 646 proteins identified as 'contaminant associated'

## 64074 features found from 6582 master proteins => contaminant features removed

## 63468 features found from 6563 master proteins => associated contaminant features removed

## 63468 features found from 6563 master proteins => MaxQuant-labelled 'Contaminants' removed

## 63468 features found from 6563 master proteins => features without a master protein removed

## 62013 features found from 6544 master proteins => features with non-unique master proteins removed

## 49175 features found from 6095 master proteins => features without quantification removed
```

Phosphosite annotation

Below, we annotate the phosphopeptide PSMs to determine the phosphosite positions and sequences.

```
proteome_fasta_filepath <- tempfile()
make_fasta(unique(phospho_infdf$Leading.razor.protein), file=proteome_fasta_filepath)
```

```
## Downloading from UniProtKB release: 2026_01
```

```
in_assay_name <- 'psm_filtered'
out_assay_name <- 'psm_annot'

qfs$phospho[[out_assay_name]] <- biomasslmb::filter_maxquant_ptm(
  qfs$phospho, in_assay_name)

# Retain phosphosites in all contexts (ptms_to_retain=c('p', '(ph)', '(ac)p'))
qfs$phospho[[out_assay_name]] <- add_filter_ptm_pos_rowdata_mq(
  qfs$phospho[[out_assay_name]], verbose = TRUE,
  min_prob = 0.501, filter_pep_by_prob = TRUE)
```

```
## The following PTMs were detected: Phospho (STY): 50438; Oxidation (M): 6067; Acetyl (Protein N-term)
```

```
# Add phospho site positions
qfs$phospho[[out_assay_name]] <- add_ptm_positions(
  obj=qfs$phospho[[out_assay_name]], proteome_fasta=proteome_fasta_filepath)

# Create a combined protein_phosphosite identifier
rowData(qfs$phospho)[[out_assay_name]]$prot_phos <- paste(
  rowData(qfs$phospho)[[out_assay_name]]$Leading.razor.protein,
```

```

gsub(' ', '', rowData(qfs$phospho)[[out_assay_name]]$ptm_name), sep='_')

# Add phospho site sequence
qfs$phospho[[out_assay_name]] <- qfs$phospho[[out_assay_name]] %>%
  add_site_sequence(proteome_fasta = proteome_fasta_filepath,
                    master_protein_col = 'Leading.razor.protein',
                    sequence_pad=7)

# Clean up the fasta file now we've finished with it
file.remove(proteome_fasta_filepath)

```

```
## [1] TRUE
```

Normalise

Next, we normalise the PSM level intensities by performing diff.median normalisation within each plex.

```

for(qf in names(qfs)){

  if(qf=='total'){
    in_assay_name <- 'psm_filtered'

  } else{
    in_assay_name <- 'psm_annot'

  }

  out_assay_name <- 'psm_filtered_norm'

  # Log2-transform, normalise, then back-transform
  qfs[[qf]][[out_assay_name]] <- QFeatures::logTransform(
    qfs[[qf]][[in_assay_name]], base=2)
  qfs[[qf]][[out_assay_name]] <- QFeatures::normalize(
    qfs[[qf]][[out_assay_name]], method='diff.median')

  assay(qfs[[qf]][[out_assay_name]]) <- 2^assay(qfs[[qf]][[out_assay_name]])

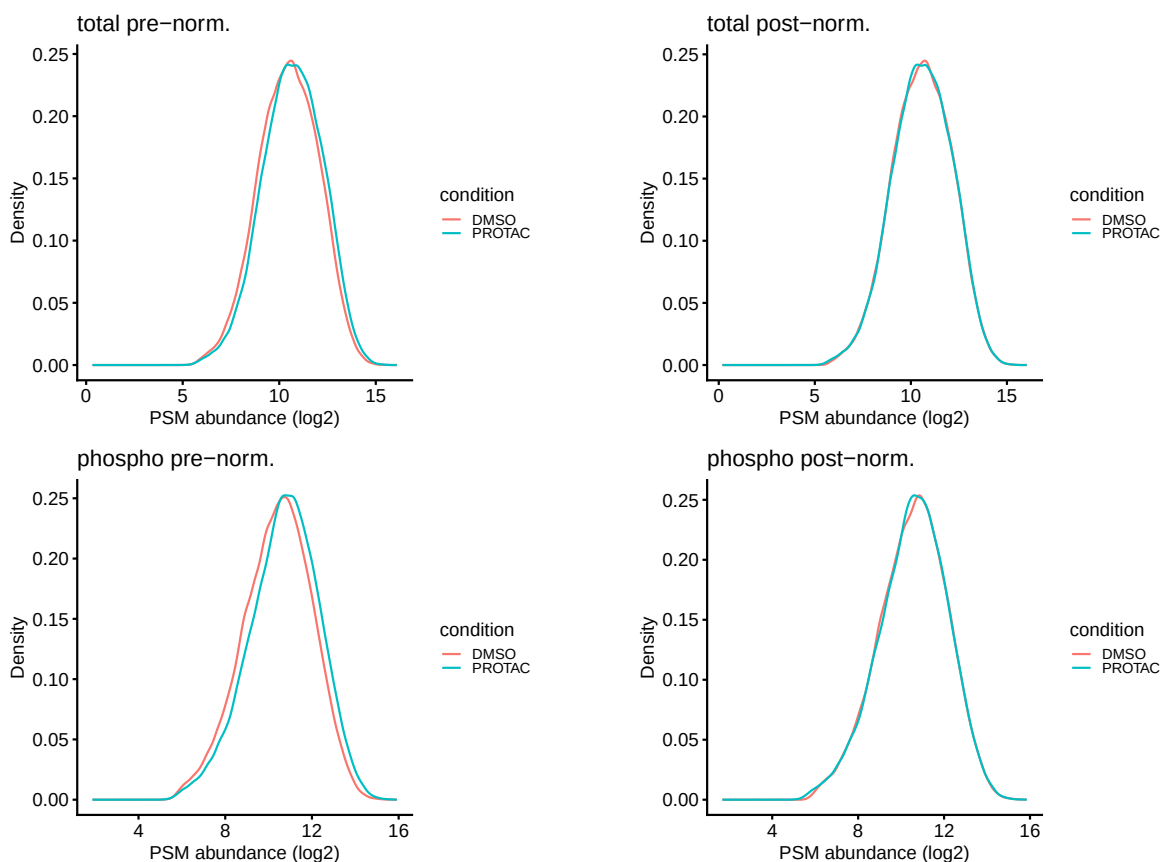
  # Plot the peptide-level quantification distributions per sample
  p <- plot_quant(qfs[[qf]][[in_assay_name]], log2transform=TRUE, method='density') +
    theme_biomasslmb() +
    xlab('PSM abundance (log2)') +
    theme(legend.key.height=unit(3, 'mm')) +
    aes(colour=condition) +
    ggtitle(paste0(qf, ' pre-norm.'))
  print(p)

  p <- plot_quant(qfs[[qf]][[out_assay_name]], log2transform=TRUE, method='density') +
    theme_biomasslmb() +
    xlab('PSM abundance (log2)') +
    theme(legend.key.height=unit(3, 'mm')) +
    aes(colour=condition) +

```

```
ggtitle(paste0(qf, ' post-norm.'))
print(p)
```

```
}
```

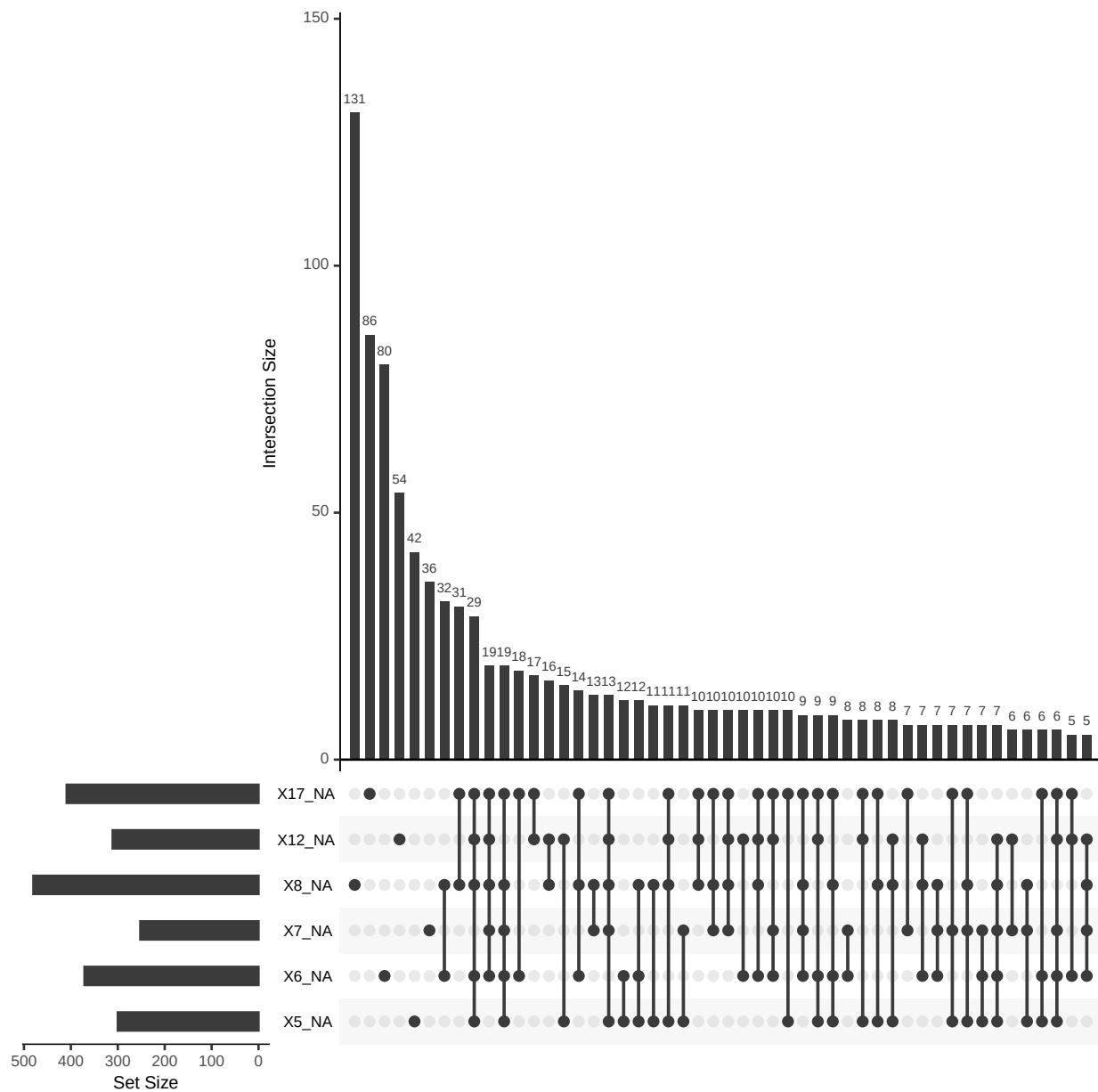


Summarising the total PSM abundances to higher level abundances

Now that we have inspected the PSM-level quantification and filtered the PSMs, we can summarise the total PSMs to protein-level abundances and the phospho PSMs to phosphosite-level abundance

For PSM to protein summarisation with TMT quantification, simply summing together the PSM-level abundances provides accurate estimates, so long as there are no missing values. Summing has the benefit of weighting the summarisation towards the most abundant PSMs, which are more likely to be accurately quantified. Where many missing values are present, the `MsCoreUtils::robustSummary` method should be preferred, since it is able to summarise accurately, even in the presence of missing values. Here, we have very few missing values, so we will use summation, with a prior step to impute the missing values.

```
plot_missing_upset(obj = qfs$total, i='psm_filtered_norm')
```



```
nNA(qfs$total, i=names(qfs$total)[grepl('_norm', names(qfs$total))])$nNA %>%
  data.frame() %>% knitr::kable()
```

assay	nNA	pNA
psm_filtered_norm	2123	0.0041176

```
nNA(qfs$phospho, i=names(qfs$phospho)[grepl('_norm', names(qfs$phospho))])$nNA %>%
  data.frame() %>% knitr::kable()
```

assay	nNA	pNA
psm_filtered_norm	1836	0.0077643

```

set.seed(42)

for(qf in names(qfs)){

  in_assay_name <- 'psm_filtered_norm'
  out_assay_name <- 'psm_filtered_forSum'

  # Remove PSMs with > 1 missing values
  qfs[[qf]][[out_assay_name]] <- QFeatures::filterNA(
    qfs[[qf]][[in_assay_name]], 1/6)

  biomasslmb::message_parse(rowData(qfs$total[[out_assay_name]]),
    'Leading.razor.protein',
    "Removing PSMs with excessive missing values")

  # Check remaining missing values
  print(sprintf('Missing values in %s = %s %%',
    qf,
    round(100*mean(is.na(assay(qfs[[qf]][[out_assay_name]]))), 3)))

  # Impute remaining missing values using MinProb. We log2-transform, impute, then back-transform
  qfs[[qf]][[out_assay_name]] <- logTransform(
    qfs[[qf]][[out_assay_name]], base=2)
  qfs[[qf]][[out_assay_name]] <- QFeatures::impute(
    qfs[[qf]][[out_assay_name]], method='MinProb')
  assay(qfs[[qf]][[out_assay_name]]) <- 2^assay(qfs[[qf]][[out_assay_name]])

}

```

```
## 85389 features found from 7729 master proteins => Removing PSMs with excessive missing values
```

```
## [1] "Missing values in total = 0.084 %"
```

```
## Loading required namespace: imputeLCMD
```

```
## Imputing along margin 2 (samples/columns).
```

```
## [1] 0.1827167
```

```
## 85389 features found from 7729 master proteins => Removing PSMs with excessive missing values
```

```
## [1] "Missing values in phospho = 0.151 %"
```

```
## Imputing along margin 2 (samples/columns).
```

```
## [1] 0.2127525
```

Finally, we perform the summarisation.

```
for(qf in names(qfs)){

  in_assay_name <- 'psm_filtered_forSum'

  if(qf=='total'){
    out_assay_name <- 'protein'
    group_col <- "Leading.razor.protein"
  } else{
    out_assay_name<- 'phosphopeptide'
    group_col <- "prot_phos"
  }

  qfs[[qf]] <- aggregateFeatures(qfs[[qf]],
                                i = in_assay_name,
                                fcol = group_col,
                                name = out_assay_name,
                                fun = base::colSums)

  # Log2-transform the summarised abundances for downstream analysis
  qfs[[qf]][[out_assay_name]] <- logTransform(qfs[[qf]][[out_assay_name]], base=2)

  qfs[[qf]][[out_assay_name]]
}
```

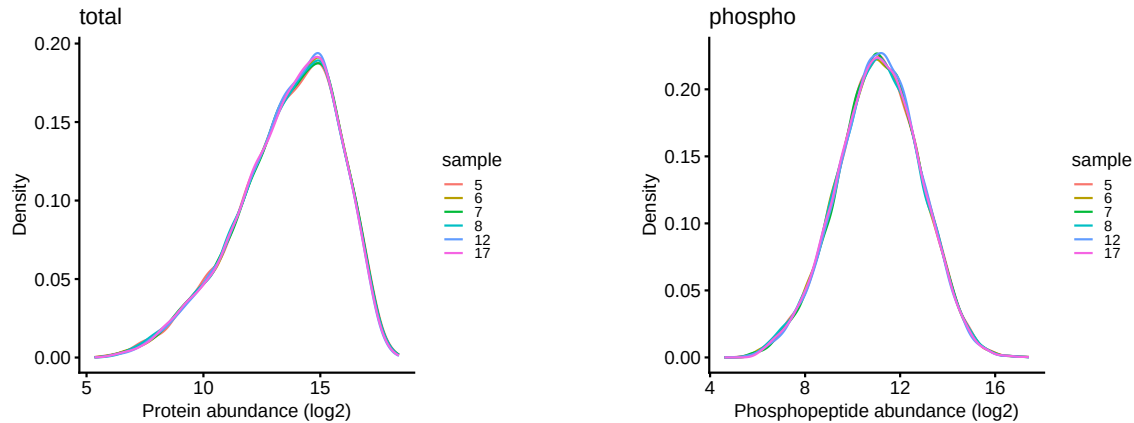
A final check that that the summarised abundances look sensible

```
for(qf in names(qfs)){

  if(qf=='total'){
    assay_name <- 'protein'
    feature <- 'Protein'
  } else{
    assay_name <- 'phosphopeptide'
    feature <- 'Phosphopeptide'
  }

  p <- plot_quant(qfs[[qf]][[assay_name]], log2transform=FALSE, method='density') +
    theme_biomasslmb() +
    xlab(sprintf('%s abundance (log2)', feature)) +
    theme(legend.key.height=unit(3, 'mm')) +
    ggtitle(qf)

  print(p)
}
```



Inspecting the number of peptides and proteins through the processing steps

Now that we have performed all the filtering steps, it's helpful to visualise how many peptides/proteins were retained at each level of the processing.

Samples per PSM

We'll start by inspecting the number of peptides in each experiment. We set the row variables to be `Sequence`, `Modifications` and `Retention.time` so that we count the number of unique PSMs.

```
for(qf in names(qfs)){

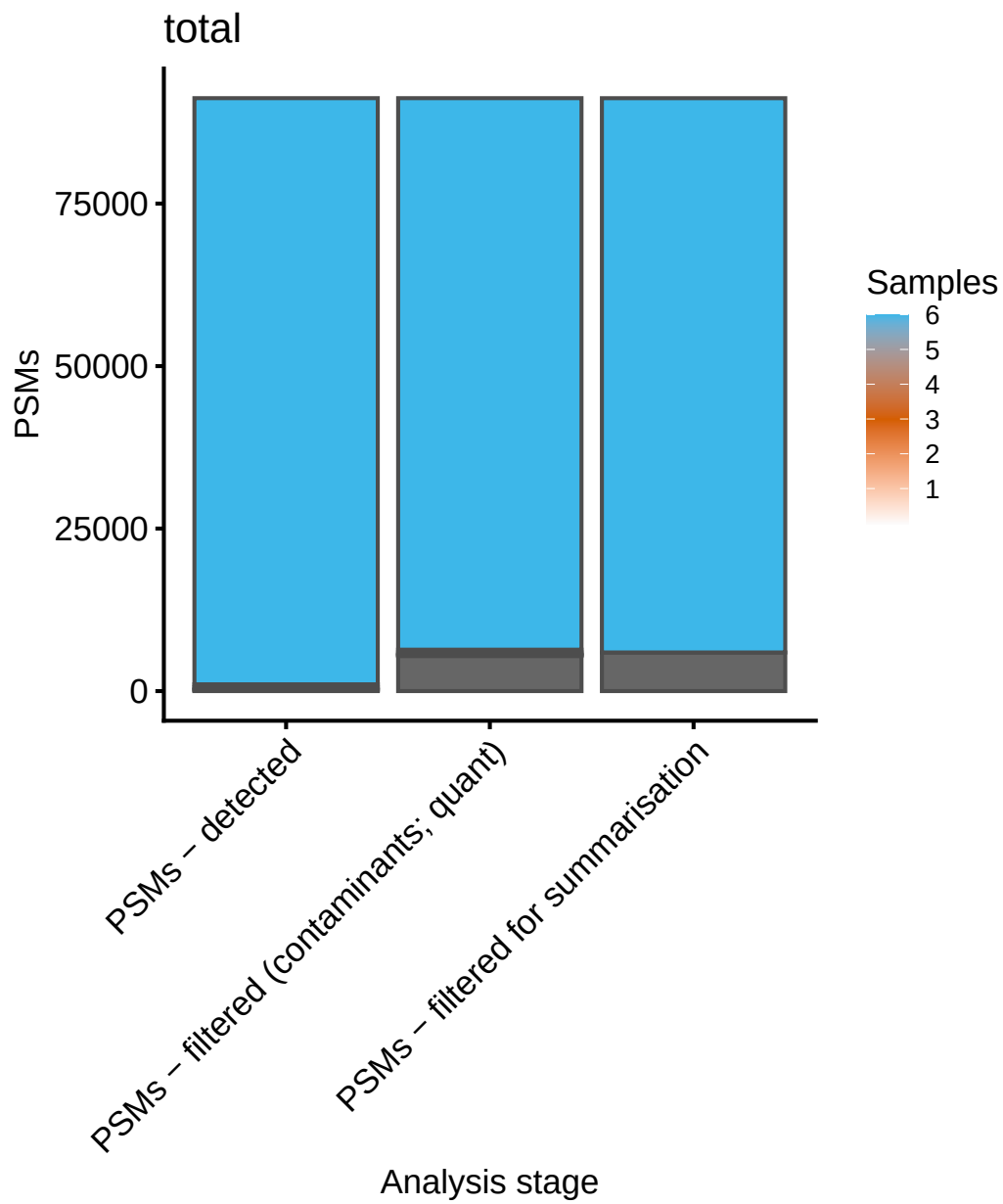
  rowvars <- c('Sequence', 'Modifications', 'Retention.time')

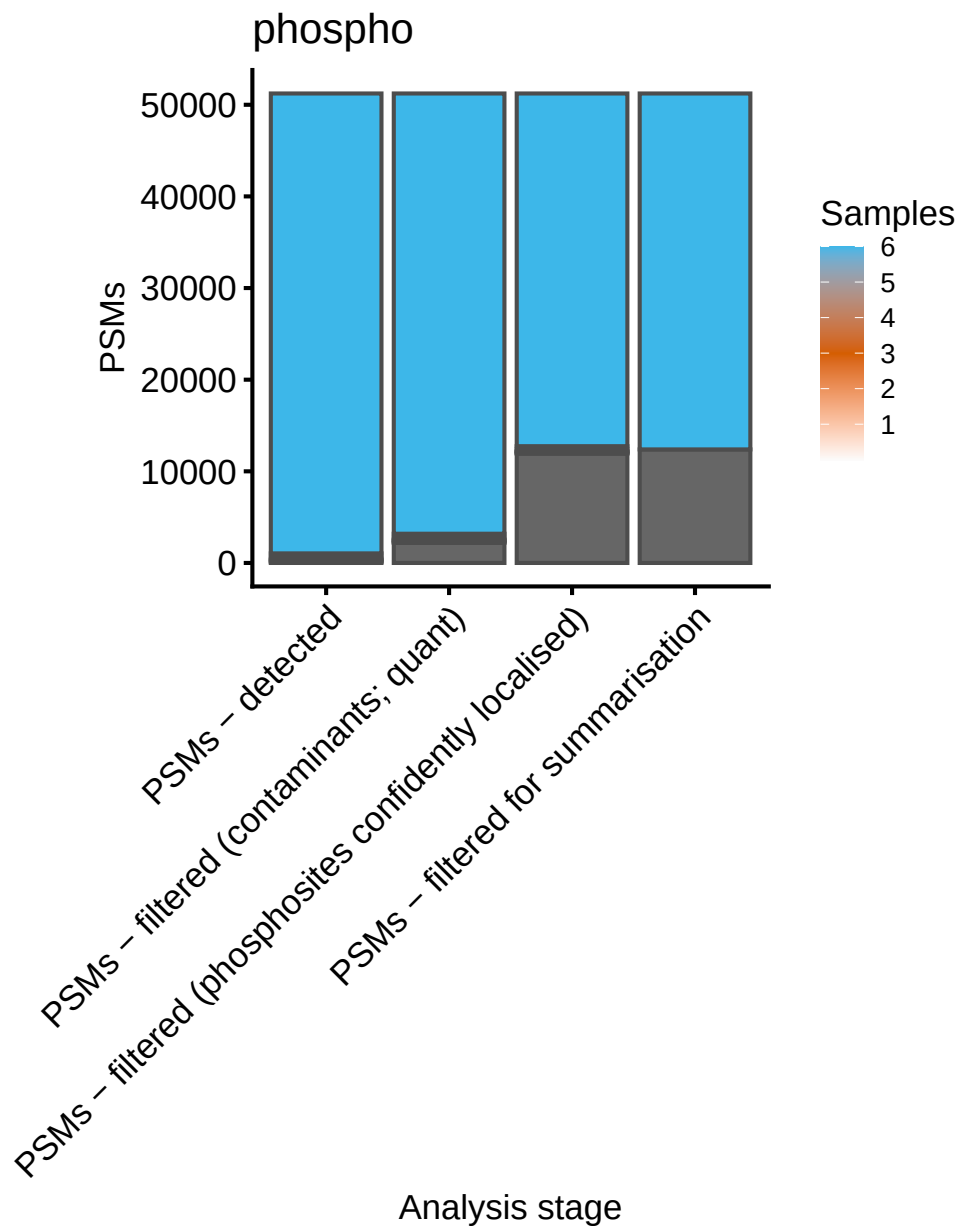
  if(qf=='total'){
    rename_cols <- c(
      'PSMs - detected' = 'psm_raw',
      'PSMs - filtered (contaminants; quant)' = 'psm_filtered',
      'PSMs - filtered for summarisation' = 'psm_filtered_forSum')
  } else{
    rename_cols <- c(
      'PSMs - detected' = 'psm_raw',
      'PSMs - filtered (contaminants; quant)' = 'psm_filtered',
      'PSMs - filtered (phosphosites confidently localised)' = 'psm_annot',
      'PSMs - filtered for summarisation' = 'psm_filtered_forSum')
  }

  samples_present <- get_samples_present(qfs[[qf]][, , unname(rename_cols)],
                                         rowvars, rename_cols)

  p <- plot_samples_present(samples_present, rowvars, breaks=seq(1,6,1)) +
    ylab('PSMs') + ggtitle(qf)

  print(p)
}
```





Samples per protein

We need to set the row variables to be just the `Leading.razor.protein` column so we count the number of unique proteins.

```
for(qf in names(qfs)){
  print(qf)

  rowvars <- c('Leading.razor.protein')

  if(qf=='total'){
    feature <- 'Proteins'
```

```

rename_cols <- c(
  'PSMs - detected' = 'psm_raw',
  'PSMs - filtered (contaminants; quant)' = 'psm_filtered',
  'PSMs - filtered for summarisation' = 'psm_filtered_forSum',
  'Protein' = 'protein')
} else{
  feature <- 'Phosphopeptides'

  rename_cols <- c(
    'PSMs - detected' = 'psm_raw',
    'PSMs - filtered (contaminants; quant)' = 'psm_filtered',
    'PSMs - filtered (phosphosites confidently localised)' = 'psm_annot',
    'PSMs - filtered for summarisation' = 'psm_filtered_forSum',
    'Phosphopeptide' = 'phosphopeptide')

}

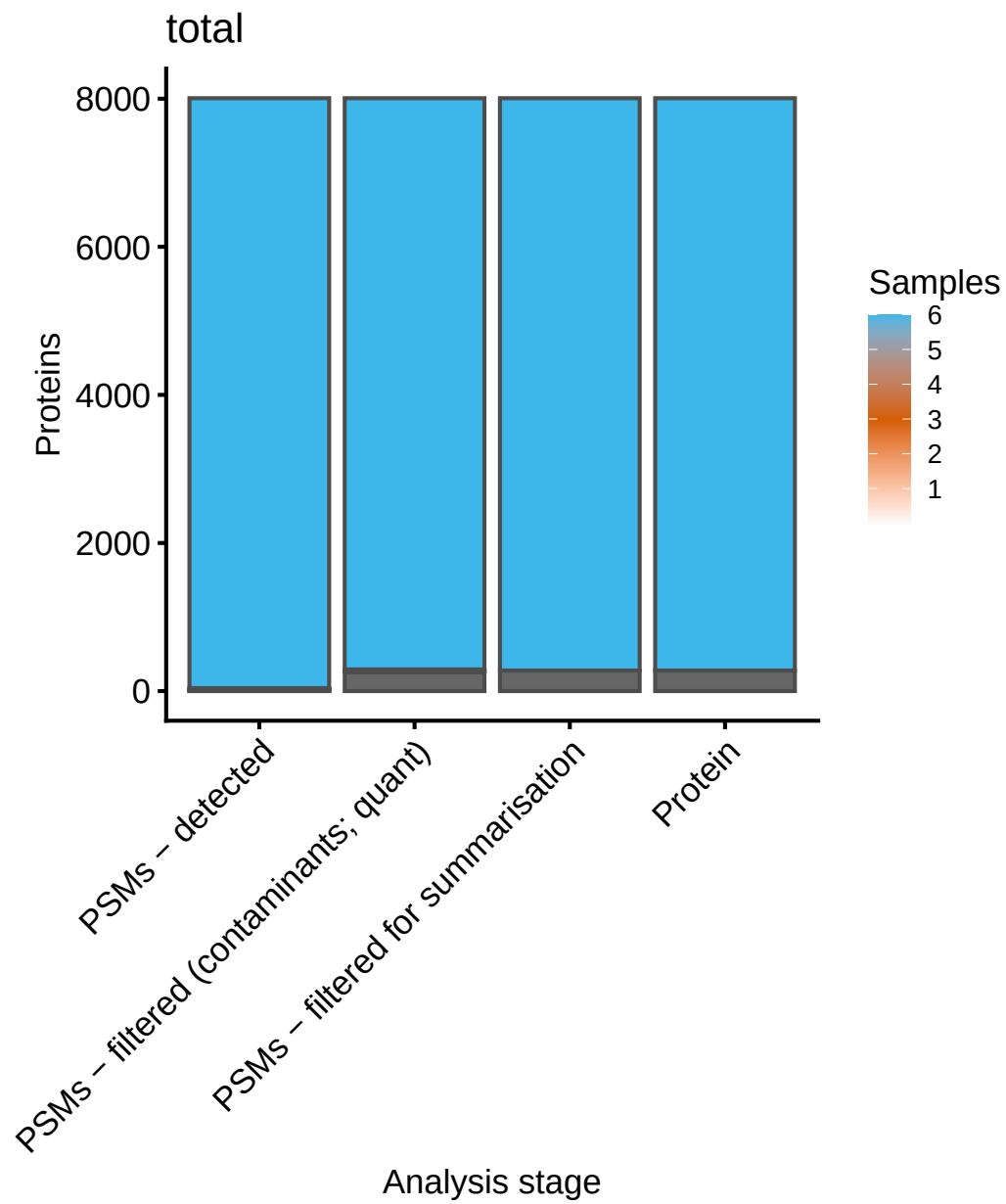
samples_present <- get_samples_present(qfs[[qf]][, , unname(rename_cols)],
                                     rowvars, rename_cols)

p <- plot_samples_present(samples_present, rowvars, breaks=seq(1,6,1)) +
  ylab(feature) + ggtitle(qf)

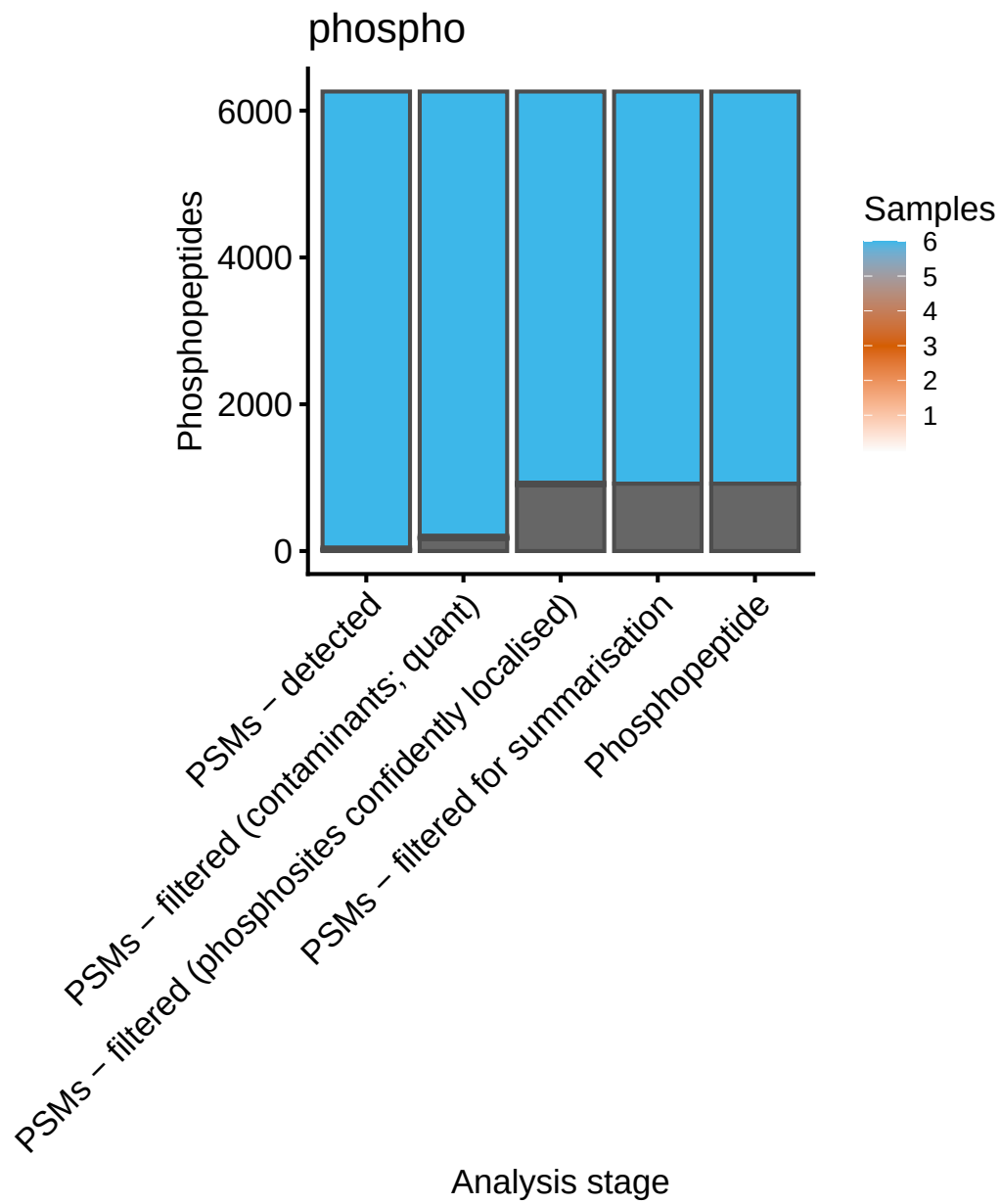
print(p)
}

```

```
## [1] "total"
```



[1] "phospho"



Saving out results

Save out the QFeatures for downstream notebooks

```
output_dir <- here('notebooks/output')

saveRDS(qfs, here(output_dir, 'PROTAC_qfs.rds'))
```

Save the protein quantification in wide format

```

protein_wide_format <- assay(qfs$total[['protein']])

# Set column names to be condition_replicate
colnames(protein_wide_format) <- sprintf(
  '%s_%s', colData(qfs$total[['protein']])$condition,
  colData(qfs$total[['protein']])$replicate)

# Order columns alphabetically
protein_wide_format <- protein_wide_format[,order(colnames(protein_wide_format))]

protein_wide_format <- protein_wide_format %>%
  data.frame() %>%
  tibble::rownames_to_column('UniprotID')

write.table(protein_wide_format,
  file = here('results/PROTAC_wide_format_protein.tsv'),
  quote=FALSE, row.names=FALSE, sep='\t')

```

We will also want the quantification in ‘tidy’ (long) form in a data.frame for plotting. Below, we create and save this

```

# Given a QFeatures, with protein level abundances,
# condition and cell line colVars and
# Master.Protein.gene.name, Master.Protein.long.name and
# Master.Protein.Descriptions Rowvars, return the abundances in a long format
# data.frame
get_long_format_exp <- function(
  obj,
  colvars=c('condition', 'replicate'),
  rowvars=c('Leading.razor.protein', 'Protein.names.y',
    'Gene.Names', 'Gene.Names.First')){

  long_format_protein <- longFormat(
    obj,
    colvars=colvars,
    rowvars=rowvars) %>%
    data.frame() %>%
    select(rowname,
      abundance=value,
      all_of(colvars),
      all_of(rowvars))

  return(long_format_protein)
}

long_format_protein <- get_long_format_exp(qfs$total[,,'protein']) %>%
  dplyr::rename('UniprotID'=rowname)

saveRDS(long_format_protein, here(output_dir, 'PROTAC_long_format_protein.rds'))

```

```

long_format_protein %>%
  select(UniprotID, Gene.Names, Gene.Names.First, Protein.names=Protein.names.y,
         condition, replicate, abundance) %>%
  write.table(file = here('results/PROTAC_long_format_protein.tsv'),
             quote=FALSE, row.names=FALSE, sep='\t')

long_format_phospho <- get_long_format_exp(
  qfs$phospho[,,'phosphopeptide'],
  rowvars=c('Leading.razor.protein', 'Protein.names.y', 'Gene.Names',
            'Gene.Names.First', 'ptm_positions_prot', 'prot_phos', 'ptm_name'))%>%
  dplyr::rename('phosphopeptide'=rowname)

saveRDS(long_format_phospho, here(output_dir, 'PROTAC_long_format_phospho.rds'))

long_format_phospho %>%
  select(UniprotID=Leading.razor.protein, phosphopeptide, ptm_positions_prot,
         Gene.Names, Gene.Names.First, Protein.names=Protein.names.y,
         condition, replicate, abundance) %>%
  write.table(file = here('results/PROTAC_long_format_phospho.tsv'),
             quote=FALSE, row.names=FALSE, sep='\t')

```

Normalising phosphoproteomics to protein abundances

Below, we create a long format data.frame containing the phosphopeptide abundances normalised to the corresponding protein abundance.

```

long_format_phosphopeptide_norm_to_protein <- long_format_protein %>%
  select(UniprotID, condition, replicate, abundance) %>%
  merge(long_format_phospho,
        by.x=c('UniprotID', 'condition', 'replicate'),
        by.y=c('Leading.razor.protein', 'condition', 'replicate'),
        suffixes=c('.protein', '.phosphopeptide')) %>%
  mutate(
    abundance_phosphopeptide_prot_norm=abundance.phosphopeptide-abundance.protein)

saveRDS(long_format_phosphopeptide_norm_to_protein,
      here(output_dir, 'PROTAC_long_format_phosphopeptide_norm_to_protein.rds'))

long_long_format_phosphopeptide_norm_to_protein <- long_format_phosphopeptide_norm_to_protein %>%
  pivot_longer(cols=matches('abundance'), names_to='type', values_to='abundance') %>%
  mutate(type=gsub('abundance.', '', type)) %>%
  mutate(type=factor(type, levels=c('protein', 'phosphopeptide',
                                    'phosphopeptide_prot_norm'))))

saveRDS(long_long_format_phosphopeptide_norm_to_protein,
      here(output_dir, 'PROTAC_long_long_format_phosphopeptide_norm_to_protein.rds'))

sessionInfo()

```

```
## R version 4.5.2 (2025-10-31)
```

```
## Platform: aarch64-apple-darwin20
## Running under: macOS Sonoma 14.7
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/vecLib.framework/Versions/A/vecLib.dylib; LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK v
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] stats4 stats graphics grDevices utils datasets methods
## [8] base
##
## other attached packages:
## [1] openxlsx_4.2.8 QFeatures_1.18.0
## [3] MultiAssayExperiment_1.34.0 SummarizedExperiment_1.38.1
## [5] Biobase_2.68.0 GenomicRanges_1.60.0
## [7] GenomeInfoDb_1.44.2 IRanges_2.42.0
## [9] S4Vectors_0.48.0 BiocGenerics_0.54.0
## [11] generics_0.1.4 MatrixGenerics_1.20.0
## [13] matrixStats_1.5.0 here_1.0.2
## [15] dplyr_1.1.4 tidyr_1.3.1
## [17] ggplot2_4.0.0 biomasslmb_0.0.4
##
## loaded via a namespace (and not attached):
## [1] DBI_1.2.3 gridExtra_2.3 sandwich_3.1-1
## [4] rlang_1.1.6 magrittr_2.0.4 clue_0.3-66
## [7] cleaver_1.46.0 compiler_4.5.2 RSQLite_2.4.3
## [10] png_0.1-8 vctrs_0.6.5 reshape2_1.4.4
## [13] stringr_1.5.2 ProtGenerics_1.40.0 pkgconfig_2.0.3
## [16] crayon_1.5.3 fastmap_1.2.0 backports_1.5.0
## [19] XVector_0.48.0 labeling_0.4.3 rmarkdown_2.29
## [22] UCSC.utils_1.4.0 UpSetR_1.4.0 visdat_0.6.0
## [25] purrr_1.1.0 bit_4.6.0 xfun_0.54
## [28] cachem_1.1.0 jsonlite_2.0.0 gmm_1.9-1
## [31] blob_1.2.4 DelayedArray_0.34.1 cluster_2.1.8.1
## [34] R6_2.6.1 stringi_1.8.7 RColorBrewer_1.1-3
## [37] genefilter_1.90.0 Rcpp_1.1.0 knitr_1.50
## [40] zoo_1.8-14 BiocBaseUtils_1.10.0 Matrix_1.7-4
## [43] splines_4.5.2 igraph_2.1.4 tidyselect_1.2.1
## [46] rstudioapi_0.17.1 abind_1.4-8 yaml_2.3.10
## [49] curl_7.0.0 lattice_0.22-7 tibble_3.3.0
## [52] plyr_1.8.9 withr_3.0.2 KEGGREST_1.48.1
## [55] S7_0.2.0 tmvtnorm_1.7 evaluate_1.0.5
## [58] uniprotREST_1.0.0 survival_3.8-3 norm_1.0-11.1
## [61] zip_2.3.3 Biostrings_2.76.0 pillar_1.11.1
## [64] corrplot_0.95 checkmate_2.3.3 rprojroot_2.1.1
## [67] scales_1.4.0 xtable_1.8-4 glue_1.8.0
## [70] lazyeval_0.2.2 tools_4.5.2 robustbase_0.99-6
## [73] annotate_1.86.1 imputeLCMD_2.1 mvtnorm_1.3-3
```

## [76]	XML_3.99-0.19	grid_4.5.2	impute_1.82.0
## [79]	MsCoreUtils_1.20.0	AnnotationDbi_1.70.0	GenomeInfoDbData_1.2.14
## [82]	nanjar_1.1.0	cli_3.6.5	S4Arrays_1.8.1
## [85]	AnnotationFilter_1.32.0	pcaMethods_2.0.0	gtable_0.3.6
## [88]	DEoptimR_1.1-4	digest_0.6.37	SparseArray_1.8.1
## [91]	farver_2.1.2	memoise_2.0.1	htmltools_0.5.8.1
## [94]	lifecycle_1.0.4	httr_1.4.7	bit64_4.6.0-1
## [97]	MASS_7.3-65		